Started on	Sunday, 14 January 2024, 8:04 PM
State	Finished
Completed on	Sunday, 14 January 2024, 8:24 PM
Time taken	19 mins 49 secs
Grade	Not yet graded

Question **1**

Complete

Marked out of 1.00

De este Rust considerat un limbaj sigur? Exemplificari raspunsul.

Spre deosebire de C, unde pointerii pot accesa o zona concreta de memorie (RAM), in Rust, nu prea merge asa ceva. Pointerii sunt ganditi sa limiteze aceasta functionalitate, care in unele cazuri poate fi periculoasa, deoarece implica coruperea memoriei.

Legat de memorie, un dezavantaj de securitate major pe care C il are este legat de Siruri de caractere. Daca punem in C mai multe caractere decat ne definim, avem buffer-overflow: se poate accesaza o zona nedorita de memorie (lucrul pe care niciodata nu ar trebui sa il vrem, decat daca suntem hackeri si exploatam memoria), iar in cel mai bun caz, crapa programul. Rust vine cu un tip de date mult mai sigur pentru siruri de caractere: `String`. Noi ca programatori nu mai trebuie sa ne facem griji de cate caractere am definit, pentru ca nu trebuie sa specificam dimensiunea. Variabila String poate avea oricate caractere (Unicode, adica si diacritice / emoji-uri)

Un alt aspect importat este sat de Mutex, Arc si Arc, care fac posibila existenta mai multor fire de executie, fara data-racing (2+ threaduri sa modifice aceasi variabila).

Propagarea eroare se face oarecum cu try-except-ul din Python, dand match pe eroare. O functie poate esua cu Err fara sa opreasca executia programului

Question 2

Complete

Marked out of 1.00

Care este rolul notiunii de lifetime in Rust? Dati exemple.

Lifetime inseamna gestionarea memorie si asiguarea sigrantei in timpul executarii programului. Lifetime are leg. cu intrebarea de dinainte, pentru ca se refera la durata pe care o "are" o variabila in memorie

Lifetime sunt utilizate pentru a specifica durata de viata a unei varaibile.

Lifetime sunt de doua tipuri:

- `a = lifetime explicit = durata de viata este in functie de context si poate fi limitata

(la un moment de timp, aceste variabile pot fi "drop"-ate, adica sa nu mai fie in RAM)

- static 'a = lifetime static = variabila va exista pe toata durata executiei programului

```
// explicit lifetime
struct Foo {
x: i32,
}

// the parameter foo and return value share the same lifetime
fn do_something<'a>(foo: &'a Foo) -> &'a i32 {
return &foo.x;
}

fn main() {
let mut foo = Foo { x: 42 };
let x = &mut foo.x;
*x = 13;
// x is dropped here, allowing us to create a non-mutable reference
let y = do_something(&foo);
println!("{}", y);
// y is dropped here
// foo is dropped here
}
```

```
// static lifetime
static PI: f64 = 3.1415;

fn main() {
// static variables can also be scoped to a function
static mut SECRET: &'static str = "swordfish";

// string literals have a 'static lifetime
let msg: &'static str = "Hello World!";
let p: &'static f64 = &PI;
println!("{} {}", msg, p);

// You can break some rules, but you must be explicit
unsafe {
// we can set SECRET to a string literal because it is also `static
SECRET = "abracadabra";
println!("{}", SECRET);
}
}
```

Question 3

Complete

Marked out of 1.00

De ce sunt utile tipurile generice? Explicati T: Trait vs &dyn Trait.

Acestea permit crearea de structuri si functii care pot lucra cu mai multe tipuri de date, orefrind astfel flexibilitate si reutilizarea a codului. Ele permit scrierea de cod mai putin, care sa acopere mai multe tipuri tipuri de date, deodata.

In loc sa scriem cod pentru fiecare tip de date in parte, definim un struct care sa le contina pe toate si un trait (interfata din Java: contine cel putin o metoda neimplementata).

```
fn aduna<T>(a: T, b: T) -> T
where     // se asteapta sa primeasca un trait
   T: std::ops::Add<Output = T>,
{
   return (a + b); // merge pentru (a, b) ap u8, f64, i32 etc.. (numere)
}
```

&dyn Trait => lucram cu trait-urile in mod dinamic, mai exact cu o instanta a unei structuri, cand nu se stie exact ce trait se foloseste pt structura. Exemplu:

&dyn = referinta imutabila la o instanta a unei structuri

```
struct SeaCreature {
pub name: String,
noise: String,
}

impl SeaCreature {
pub fn get_sound(&self) -> &str {
&self.noise
}
}

trait NoiseMaker {
fn make_noise(&self);
}

impl NoiseMaker for SeaCreature {
fn make_noise(&self) {
println!("{}", &self.get_sound());
}
}

fn static_make_noise(creature: &SeaCreature) {
// we know the real type
creature.make_noise();
}

fn dynamic_make_noise(noise_maker: &dyn NoiseMaker) {
// we don't know the real type
noise_maker.make_noise();
}

fn main() {
let creature = SeaCreature {
```

```
name: String::from("Ferris"),
noise: String::from("blub"),
};
static_make_noise(&creature);
dynamic_make_noise(&creature);
}
```